

Einleitung & Projektbezug

Bei dem rundenbasierten Multiplayer-Spiel *Phantom Hunt* treten drei Geister gegen einen Menschen an. Während der Mensch versucht, dem Spielfeld zu entkommen, wollen die Geister ihn erschrecken. Das Spiel läuft über vier Runden, wobei die Rolle des Menschen rotiert, sodass jeder Spieler genau einmal in dieser Position spielt.

Die Serverarchitektur basiert auf einem Thread-per-Client-Modell mit TCP-Sockets. Jedem Client wird ein eigener Thread zugewiesen. Spielzustände und Nicknames werden zentral verwaltet, die Kommunikation erfolgt über ein textbasiertes Protokoll.

Wenn mehrere Threads gleichzeitig auf die gleiche *Registry* zugreifen, muss die Thread-Safety gewährleistet sein. Anderenfalls können unkontrollierte Zugriffe zu Datenkorruption führen. Ausserdem erfordert das zeitbasierte Scoring eine konsistente Synchronisation von Timern und Punkteständen zwischen Server und allen Clients. Selbst kleinste Abweichungen würden die Spielfairness gefährden. Schliesslich hat die Qualitätssicherung die Aufgabe, Fehler zu verhindern. Sie fängt Abstürze durch fehlerhafte oder unerwartete Netzwerkpakete zuverlässig ab.

Hierarchie der Messung (FCM-Modell)

In *Phantom Hunt* werden Punkte kontinuierlich sowie ereignisbasiert vergeben. Fehler in der Berechnung wirken sich direkt auf den Spielausgang aus und gefährden die Fairness. Die Qualität hängt deshalb wesentlich von der Korrektheit der Scoring-Logik im GameHandler ab. Gemessen wird dies über die Zeilenabdeckung (Line Coverage) mit JaCoCo.

Da *Phantom Hunt* auf einem textbasierten Protokoll und einer Thread-per-Client-Architektur basiert, muss das System fehlerhafte Eingaben sowie Verbindungsabbrüche stabil verarbeiten. Die Fehlertoleranz beschreibt, wie gut ungültiger Input abgefangen wird und wie zuverlässig Ressourcen bereinigt werden. Zur Messung werden zwei Metriken eingesetzt. Einerseits die Abdeckung ungültiger Äquivalenzklassen, welche überprüft ob fehlerhafte Protokoll-Strings korrekt behandelt werden. Andererseits die Socket-Cleanup-Erfolgsrate, die misst, ob Verbindungen schnell und vollständig freigegeben werden.

Die Architektur ist modular in Lobby, Game, Net und Registry aufgebaut, wodurch Verantwortlichkeiten klar getrennt sind. Für eine effiziente Fehlersuche muss der Code überschaubar bleiben. Die Analysierbarkeit wird über die zyklomatische Komplexität $v(G)$ mit Metrics Reloaded gemessen. Um die Wartbarkeit sicherzustellen, wird für zentrale Methoden ein Grenzwert von $v(G) \leq 4.0$ definiert.

QA-Massnahmen

Zur Fehlervermeidung wird im GameHandler Design by Contract eingesetzt, indem vor kritischen Aktionen der Spielzustand mittels Assertions überprüft wird. Dadurch werden ungültige Zustände früh erkannt. Zusätzlich wird im Netzwerkpaket try-with-resources verwendet, um sicherzustellen, dass Ressourcen auch im Fehlerfall korrekt geschlossen werden.

Auf organisatorischer Ebene werden zentrale Schnittstellen mit Javadoc dokumentiert, um Missverständnisse zu vermeiden. Für besonders kritische Komponenten wie die Registry wird Pair Programming eingesetzt, wodurch parallele Fehlerquellen reduziert werden.

Vor dem Merge in den Main-Branch wird der Code durch Reviews geprüft, insbesondere hinsichtlich Threadsicherheit und Lesbarkeit. Parallel dazu wird die Komplexität zentraler Klassen kontinuierlich überwacht, um strukturelle Probleme früh zu erkennen.

Im Bereich der dynamischen Tests liegt der Fokus auf Unit-Tests mit JUnit für GameHandler und Protocol. JaCoCo misst dabei die Testabdeckung. Ergänzend werden Black-Box-Tests eingesetzt, bei denen Eingaben systematisch in gültige und ungültige Äquivalenzklassen unterteilt werden.

Messungen & Planung

Line Coverage

JaCoCo wird ab MS4 eingesetzt, während für MS2 keine Messwerte vorliegen und die Abdeckung mit 0% definiert ist. Aktuell liegt die durchschnittliche Coverage im Client-Core bei etwa 27%, mit Spitzenwerten bis 49%. Kritische Backend-Pakete wie `server.game` und `common.protocol` weisen noch eine Abdeckung von 0% auf.

Zur besseren Übersicht wurden die Pakete in fünf Architekturschichten zusammengefasst (Client-Core, Game-Logik, Protokoll, Server-Netzwerk und Frontend). Die Ergebnisse zeigen klaren Handlungsbedarf im Backend.

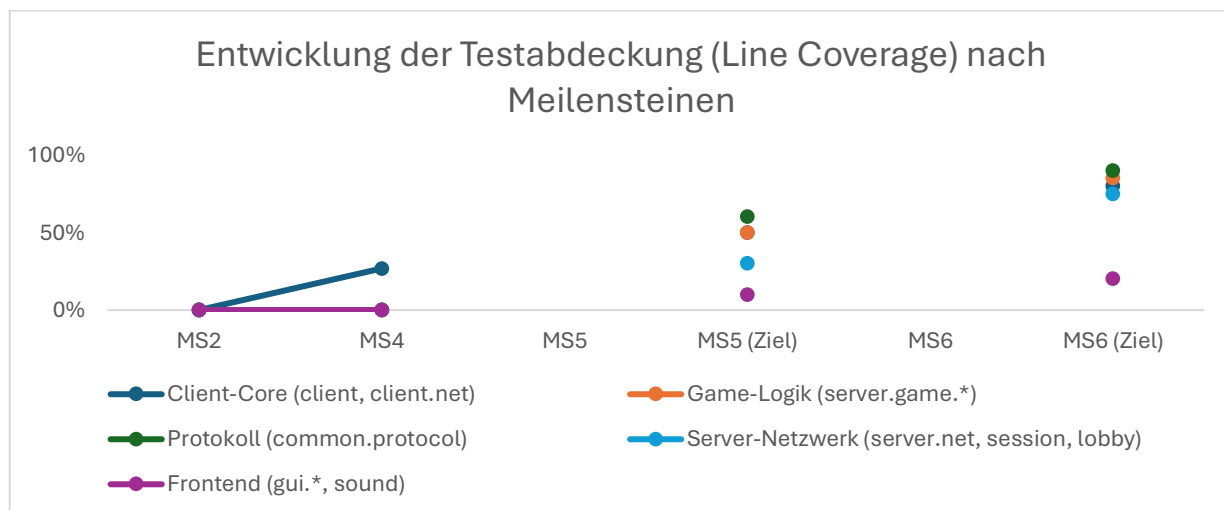


Abbildung 1: Testabdeckung pro Architekturschicht (MS4–MS6)

Zyklomatische Komplexität

Die durchschnittliche zyklomatische Komplexität liegt in allen Schichten zwischen 1.46 und 2.47 und damit deutlich unter dem Grenzwert von 4.0. Dies zeigt, dass die aktuelle Struktur gut wartbar ist und keine übermäßige Zentralisierung von Logik vorliegt.

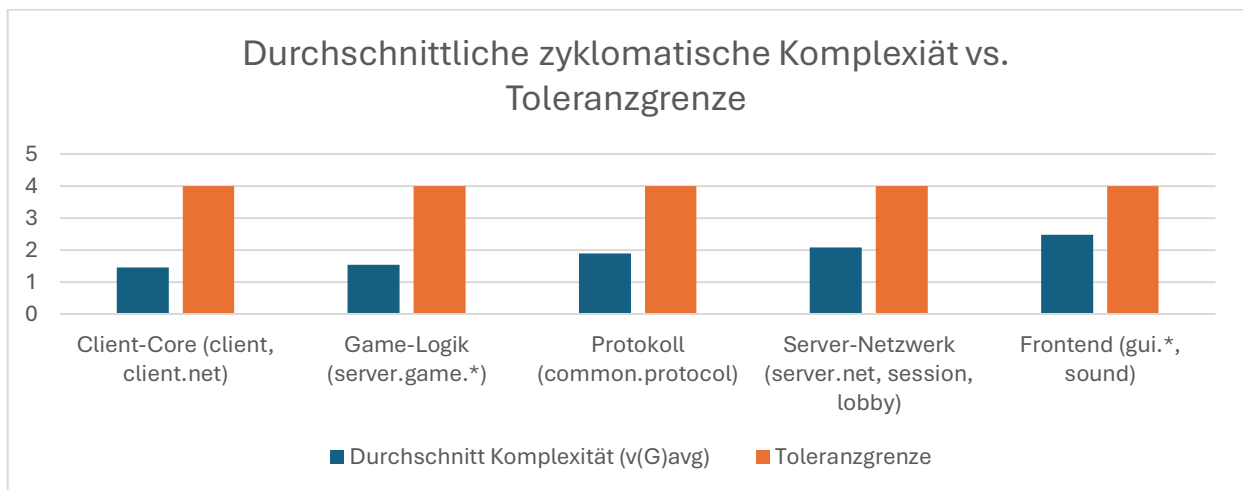


Abbildung 2: Komplexität pro Architekturschicht (MS4)

Umsetzungsplanung

Im fünften Meilenstein liegt der Fokus auf dem gezielten Ausbau der Testabdeckung. Die Coverage im game-Paket soll von aktuell 0% auf 50% erhöht werden, wobei insbesondere die Scoring-Logik abgesichert wird. Parallel dazu werden Black-Box-Tests für Netzwerkabbrüche durchgeführt und die Komplexität weiterhin überwacht.

Im sechsten Meilenstein erfolgt die abschliessende Qualitätssicherung. Die Testabdeckung in den Paketen game und protocol wird auf 85-90% erhöht. Zusätzlich wird das System durch vollständiges Durchspielen aller Runden getestet, bevor abschliessend Bugfixes, Dokumentation und Refactoring durchgeführt werden.

RACI-Matrix

Zur klaren Zuordnung der Verantwortlichkeiten im QA-Prozess wird eine RACI-Matrix eingesetzt. Sie stellt sicher, dass jede Aufgabe eindeutig einer verantwortlichen Person zugewiesen ist und verhindert Verantwortungsdiffusion im Team. Gleichzeitig unterstützt sie das Vier-Augen-Prinzip bei kritischen Komponenten.

Die Matrix fokussiert insbesondere auf die im JaCoCo-Report identifizierten Schwachstellen, vor allem in den Bereichen Game-Logik und Protokoll. Durch die klare Zuweisung wird sichergestellt, dass zentrale QA-Ziele wie die Erhöhung der Testabdeckung systematisch umgesetzt werden.

QA-Aufgabe	Silas	Jan	Hermes	Ismail	Vera
Unit-Tests GameHandler (Spiellogik)	I	A	R	I	I
Unit-Tests Protocol (Kommunikation)	I	C	R	A	I
Black-Box-Tests (Stabilität)	R	I	I	I	A
Code-Review Registry (Concurrency)	C	R	A	I	I
Javadoc & Standards (Wartbarkeit)	R	R	R	R	A

R steht für Responsible, A für Accountable, C für Consulted und I für Informed.